

Exercise 10: Linear Programming (Sparse Quantile Regression)

Optimization in Machine Learning

Reproducibility

Produced with Python 3.10 and R 4.4.1. To reproduce, install the pinned package versions below.

Python

```
pip install numpy==2.0.2 matplotlib==3.10.8 scipy==1.15.3
```

R

```
install.packages("remotes")
remotes::install_version("ggplot2", "3.5.1")
remotes::install_version("linprog", "0.9.6")
```

Setup

We have $n = 50$ univariate observations with skewed (gamma) errors:

$$x_i \sim \text{Uniform}(0, 1), \quad y_i = 2x_i + \text{Gamma}(\text{shape} = 1).$$

The skewed noise motivates *quantile* regression instead of OLS — we want to fit the conditional quantile of $y \mid x$, not the conditional mean.

Multivariate sparse quantile regression is

$$\min_{\beta \in \mathbb{R}^p} \frac{1}{n} \sum_{i=1}^n \rho_{\tau}(y^{(i)} - \beta_0 - \beta^{\top} \mathbf{x}^{(i)}) \quad \text{s.t.} \quad \|\beta\|_1 \leq t$$

with $\tau \in (0, 1)$ and the **pinball loss**

$$\rho_{\tau}(s) = \begin{cases} \tau \cdot s & s > 0 \\ -(1 - \tau) \cdot s & s \leq 0 \end{cases}$$

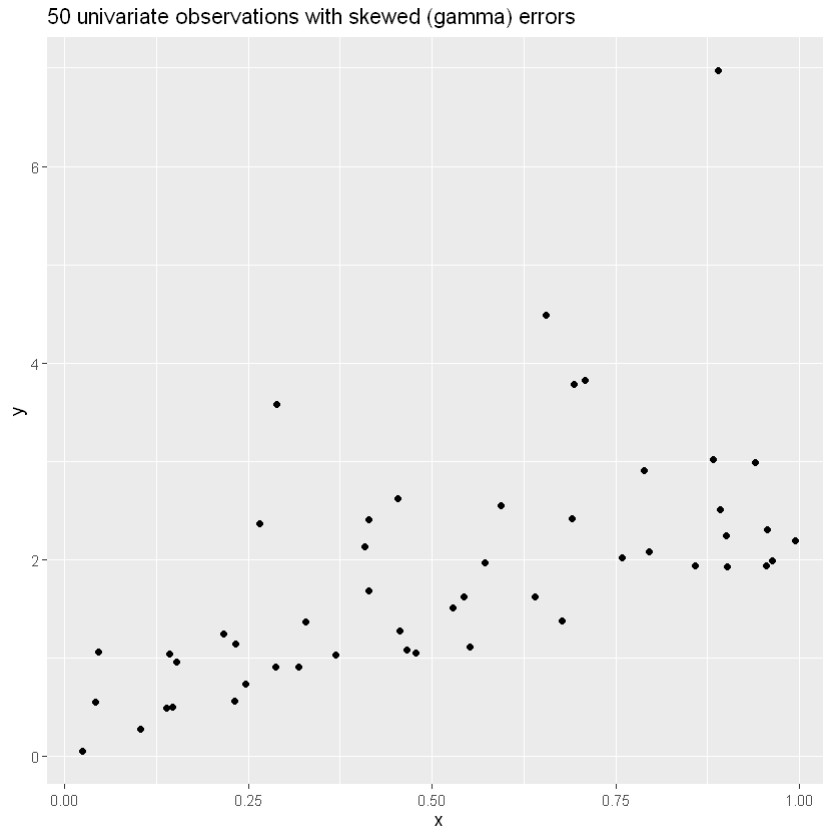
(asymmetric residual weighting; for $\tau = 0.5$ this is just $\frac{1}{2}|s|$). We treat the univariate ($p = 1$) case below.

R

```
library(ggplot2)

set.seed(123)
n <- 50
x <- runif(n)
y <- 2 * x + rgamma(n, shape = 1)

ggplot(data.frame(x = x, y = y), aes(x, y)) + geom_point() +
  ggtitle("50 univariate observations with skewed (gamma) errors")
```



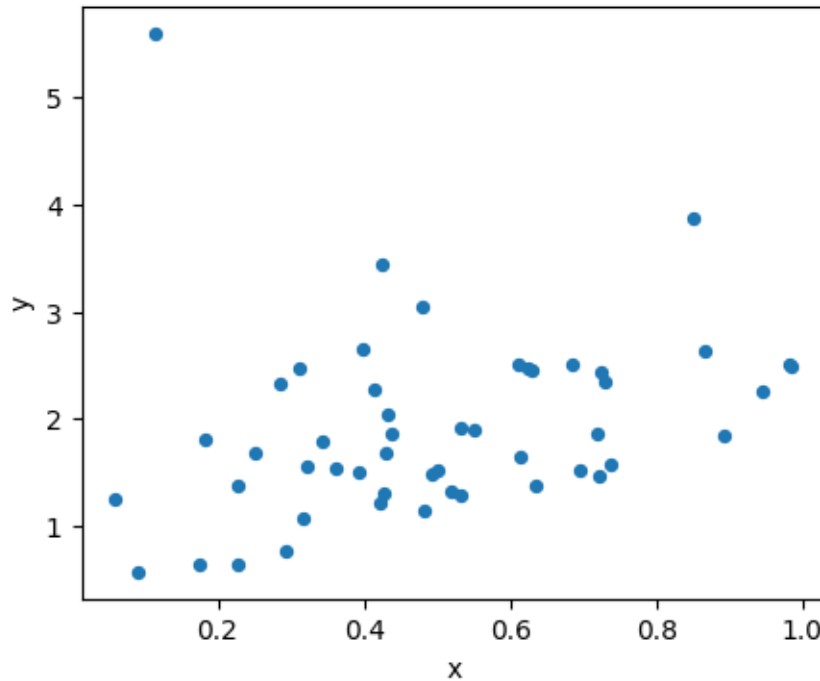
Python

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(123)
n = 50
x = np.random.uniform(size=n)
y = 2 * x + np.random.gamma(shape=1, size=n)

fig, ax = plt.subplots(figsize=(5, 4))
ax.scatter(x, y, s=18)
ax.set_xlabel("x"); ax.set_ylabel("y")
ax.set_title("50 univariate observations with skewed (gamma) errors")
plt.show()
```

50 univariate observations with skewed (gamma) errors



i Note

R vs. Python parity. R's `set.seed(123)` and Python's `np.random.seed(123)` use *different* RNGs, and `runif` / `rgamma` differ from `np.random.uniform` / `np.random.gamma` even given the same logical seed. The simulated data therefore differs between languages, so the LP solutions $\hat{\beta}$ will differ as well. Both implementations are correct; only the data differs. *Within* a language, the LP is deterministic — the same standard-form input gives identical $\hat{\beta}$ from `linprog` / `linprog`.

(a) Standard form of the LP

Find the standard LP form of the one-dimensional sparse quantile regression problem.

Hint: an unconstrained variable x can be decomposed into two non-negative variables $x = x^+ - x^-$, with $|x| = x^+ + x^-$. (At the optimum at most one of x^+, x^- is nonzero.)

(b) Visualise the empirical risk and feasible region

Plot $\mathcal{R}_{\text{emp}}(\beta_0, \beta_1)$ for $(\beta_0, \beta_1) \in [-3, 3]^2$ with $\tau = 0.4$. Mark the feasible region $|\beta_1| \leq t = 1.7$.

(c) Solve the LP

Use R's `linprog::solveLP` (or Python's `scipy.optimize.linprog`) to solve the sparse 40 % quantile regression with $t = 1.7$.

(d) Dual formulation

State the dual of the LP from (a).